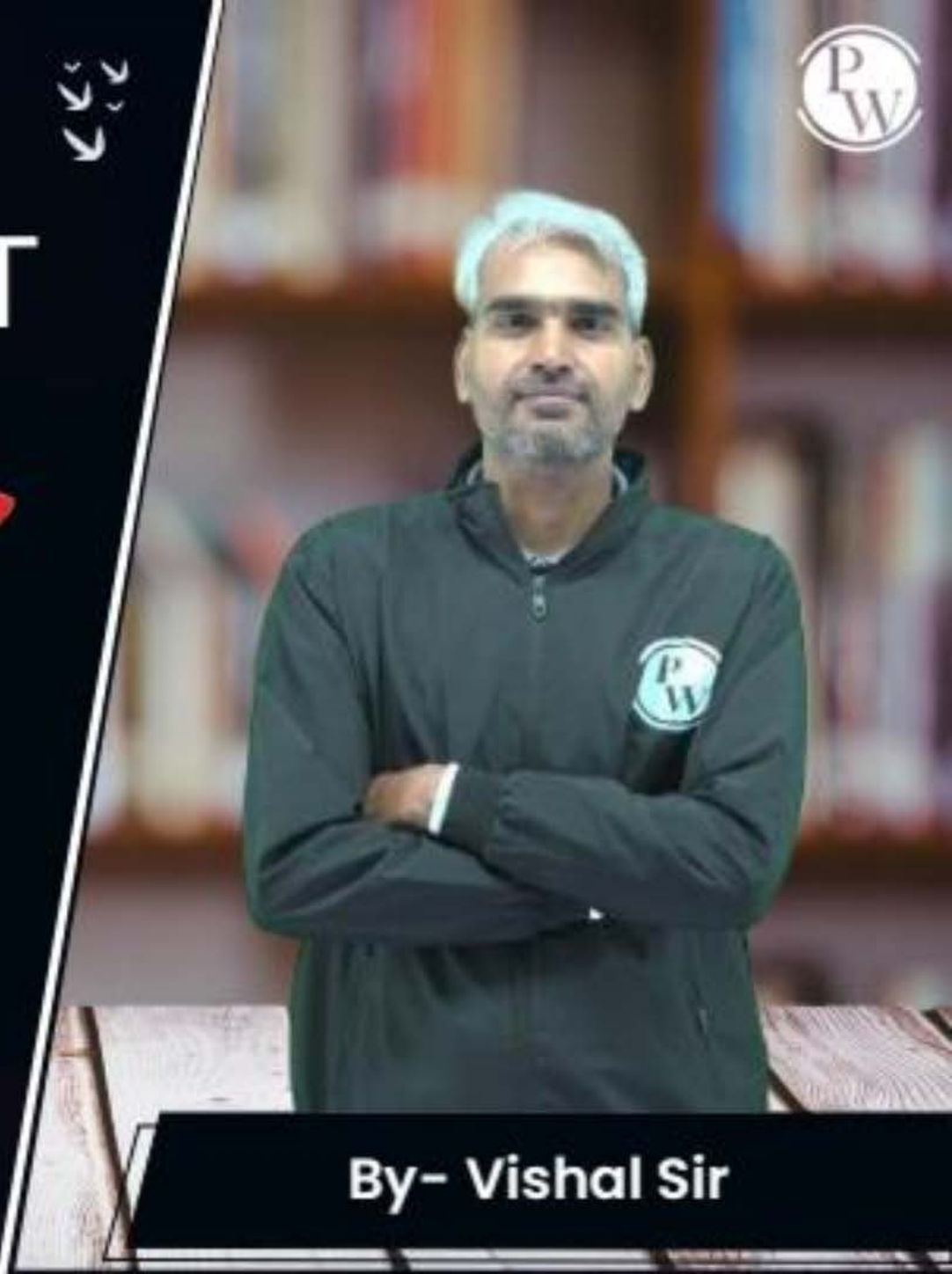
Computer Science & IT



Compiler Design

Syntax Analysis (Parser)

Lecture No. 04



By- Vishal Sir

Recap of Previous Lecture



Topic Types of Parser

Topic Elimination of Left Recursion

Slide

Topics to be Covered



Topic — Elimination of Left Recursion

Topic — Left Factoring

Topic — First and Follow

Slide

Topic : Problem with Left Recursive CFG

Top-down parser may get stuck in the infinite loop with left recursive grammar

eg:- $A \rightarrow A\alpha / \alpha$

string $w = \alpha\alpha\alpha\alpha$



eg:- $A \rightarrow \alpha A / \alpha$

string $w = \alpha\alpha\alpha\alpha$



Slide



Topic : Elimination of Left Recursion

eg $A \longrightarrow A\alpha / \beta$

Language $= \{\beta, \beta\alpha, \beta\alpha\alpha, \beta\alpha\alpha\alpha, \ldots\}$

$= \{\beta.\alpha^*\}$

infinite language

↳ to generate an infinite language recursion is required

{ but top-down parser can not work with left-recursion }

∴ We need to eliminate left recursion from the grammar, and we need to write an equivalent grammar using right recursion

Two grammars are said to be equivalent only if language generated by them is same

Slide

① $A \rightarrow A\alpha / \beta$

$L = \{\beta . \alpha^*\}$

we will try to generate α^* using Right-Recursive Production.

Let α^* is Produced Right Recursively using non-terminal A'

$L = \beta\alpha^*$

$\therefore A \rightarrow \beta A'$

$A' \rightarrow \alpha A' / \epsilon \Rightarrow L = \alpha^*$

$A \rightarrow A\alpha / \beta \Rightarrow$ eliminate left Recursion $\Rightarrow$

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' / \epsilon$

② $A \rightarrow A\alpha / \beta_1 / \beta_2 / \beta_3$

$L = \{(\beta_1 + \beta_2 + \beta_3) \cdot \alpha^*\}$ ∴

$= \{\beta_1 \cdot \alpha^* + \beta_2 \alpha^* + \beta_3 \alpha^*\}$

let α^* are produced Right recursively using A'

$L = \{\beta_1 \cdot \alpha^* + \beta_2 \alpha^* + \beta_3 \alpha^*\}$

$A \rightarrow \beta_1 A' / \beta_2 A' / \beta_3 A'$

$A' \rightarrow \alpha A' / \in$

③ $A \rightarrow A\alpha_1 / A\alpha_2 / \beta_1 / \beta_2$

$$L = \{ \beta_1 (\alpha_1+\alpha_2)^* + \beta_2 (\alpha_1+\alpha_2)^* \} = \{ (\beta_1+\beta_2) \cdot (\alpha_1+\alpha_2)^* \}$$

let $(\alpha_1+\alpha_2)^*$ is generated right recursively using A'

$$L = \{ \beta_1 (\alpha_1+\alpha_2)^* + \beta_2 (\alpha_1+\alpha_2)^* \}$$

$A \rightarrow \beta_1 \cdot A' \; / \; \beta_2 \cdot A'$

$A' \rightarrow \alpha_1 A' / \alpha_2 A' / \epsilon$

(4) $A \longrightarrow A\alpha / \beta_1 / \beta_2 / \epsilon$

$$L = \{ \beta_1 \alpha^* + \beta_2 \alpha^* + \epsilon \alpha^* \} \equiv \{ (\beta_1 + \beta_2 + \epsilon) \alpha^* \}$$

let α^* is generated using A'

$$L = \{ \beta_1 \alpha^* + \beta_2 \alpha^* + \epsilon \alpha^* \}$$

$A \longrightarrow \beta_1 A' / \beta_2 A' / \epsilon A' \equiv$

$A \longrightarrow \beta_1 A' / \beta_2 A' / A'$

$A' \longrightarrow \alpha A' / \epsilon$



Topic : Elimination of Left Recursion

eg: Eliminate left recursion from following grammar

$A \to A\alpha / \beta$

$E \to E+T / T$

$G_1 = \begin{cases} E \to E+T / T \\ T \to T*F / F \\ F \to id \end{cases}$

No left Recursion

$E \to E+T / T \equiv \begin{cases} E \to TE' \\ E' \to +TE' / \epsilon \end{cases}$

$T \to T*F / F \equiv \begin{cases} T \to FT' \\ T' \to *FT' / \epsilon \end{cases}$

$F \to id$

$A \to \beta A'$

$A' \to \alpha A' / \epsilon$

$E \to TE'$

$E' \to +TE' / \epsilon$

Slide



Topic : Elimination of Left Recursion

1. S→aBDh

B→Bb/h ⟹ $\begin{cases} B \rightarrow hB' \\ B' \rightarrow bB'/\epsilon \end{cases}$

D→EF

E→g/ϵ

F→f/ ϵ

Slide



Topic : Elimination of Left Recursion

2. $S \rightarrow (L)/a \equiv S \rightarrow (L)/a$

$L \rightarrow L, S/S \equiv \begin{cases} L \rightarrow S\,L' \\ L' \rightarrow ,S\,L'/\epsilon \end{cases}$

Slide



Topic : Elimination of Left Recursion

3. $S \rightarrow A$

$A \rightarrow Ad/Ac/aB/ac$ (β_1, β_2)

$B \rightarrow bBc/f$

$S \rightarrow A$

$A \rightarrow aBA' / acA'$

$A' \rightarrow dA' / cA' / \epsilon$

$B \rightarrow bBc/f$

Slide

Topic : Elimination of Left Recursion

$L = (c+ad)^*$

4. $S \rightarrow Aa/b$

$A \rightarrow Ac/Sd/\epsilon$

Indirect Recursion

$\Rightarrow$ Substitute S from above Production

$S \rightarrow Aa/b$

$A \rightarrow Ac/Aad/bd/\epsilon$

Eliminate left Recursion

$A \rightarrow bdA'/\epsilon A'$

$A' \rightarrow cA'/adA'/\epsilon$

Final grammar after Elimination of Left Recursion

$S \rightarrow Aa/b$

$A \rightarrow bdA'/A'$

$A' \rightarrow cA'/adA'/\epsilon$

Slide

Topic : Elimination of Left Recursion

5. $A \rightarrow Ba / Aa / c$

$B \rightarrow Bb / Ab / d$

Step-1: Eliminate left Recursion from top production

$A \longrightarrow BaA' / cA'$

$A' \longrightarrow aA' / \epsilon$

Step-2: Substitute A from newly generated production into productions of non-terminal B

$B - Bb / Ab / d$

$BaA'b \quad cA'b$

∴ Productions of B are $B \longrightarrow Bb / BaA'b / cA'b / d$

Step-3: Eliminate left Recursion from productions of B

$B \longrightarrow cA'bB' / dB'$

$B' \longrightarrow bB' / aA'bB' / \epsilon$

Final gramman

$A \rightarrow BaA' / cA'$

$A' \rightarrow aA' / \epsilon$

$B \rightarrow cA'bB' / dB'$

$B' \rightarrow bB' / aA'bB' / \epsilon$

Slide



Topic : Elimination of Left Recursion

6. $X \rightarrow \underline{X}\underline{Sb} / \underline{S}a / \underline{b}$

$S \rightarrow Sb / \underline{X}a / a$

⇓

$X \rightarrow SaX' / bX'$
$X' \rightarrow SbX' / \epsilon$
$S \rightarrow bX'aS' / aS'$
$S' \rightarrow bS' / aX'aS' / \epsilon$

Step-1 :

$X \rightarrow SaX' / bX'$
$X' \rightarrow SbX' / \epsilon$

Step-2 Substitute

$S \rightarrow Sb / SaX'a / bX'a / a$

Step-3 Remove left Recursion from :

$S \rightarrow bX'aS' / aS'$
$S' \rightarrow bS' / aX'aS' / \epsilon$

Slide



Topic : Common Prefix Problem

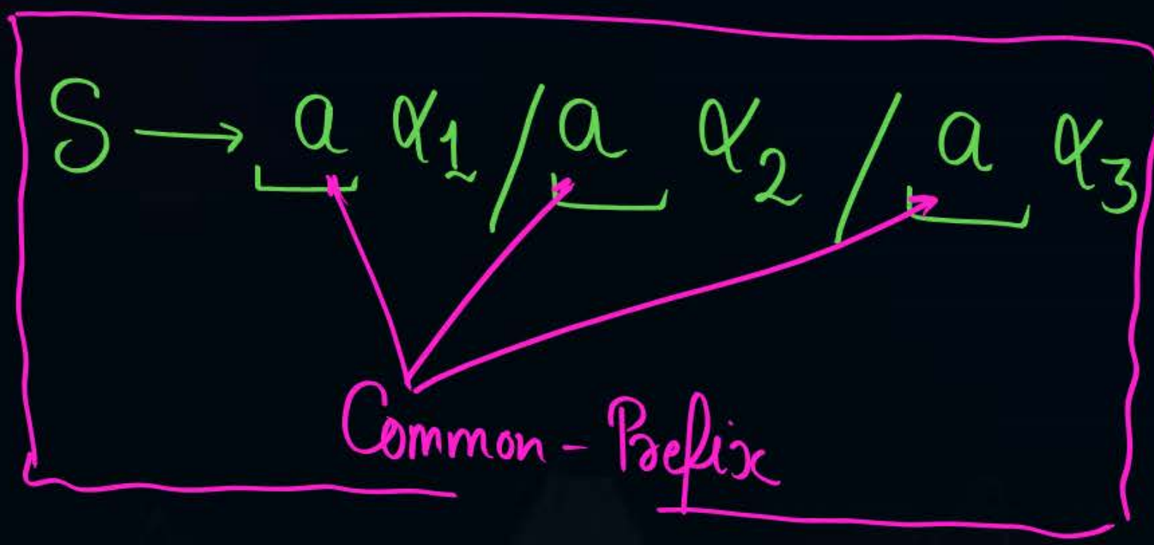
* Top-down parser can see only one symbol of input string at a time from left to right

* If grammar is

$S \rightarrow a\alpha_1 / a\alpha_2 / a\alpha_3$

and input string is,

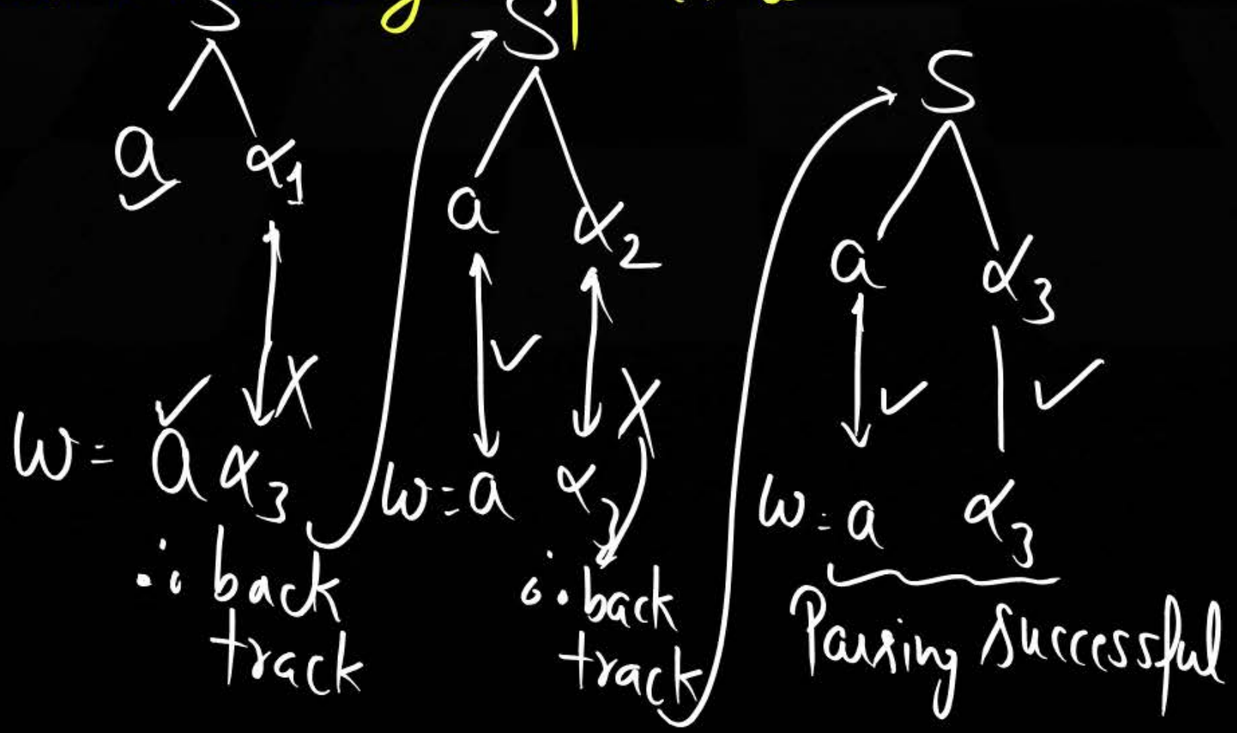
$w = a\alpha_3$



* Initially top-down parser can see symbol 'a' of input string, and it will look for the production that can generate 'a' as first symbol.

Slide

* All three productions can generate 'a' as first symbol, ∴ Parser can not choose the right production {i.e., $S \rightarrow a\alpha_3$} deterministically
∴ It may have to backtrack, and backtracking will result in wastage of time.





Topic : Non-deterministic grammar

A grammar with Common-prefix is called Non-deterministic grammar.

Slide



Topic : Left Factoring

Elimination of Common-prefix | Conversion of non-deterministic grammar into deterministic grammar

* Left factoring is the process of eliminating the common prefix, resulting grammar is called left factored grammar.

{ We will try to write an equivalent grammar which is free from common prefix }

Slide

eg $\{S \rightarrow a\alpha_1 / a\alpha_2 / a\alpha_3$

Language $= \{ a(\alpha_1 + \alpha_2 + \alpha_3) \}$

Let a new non-terminal S' is used to generate $(\alpha_1 + \alpha_2 + \alpha_3)$

$\therefore \quad S \rightarrow a\alpha_1 / a\alpha_2 / a\alpha_3 \equiv$

$S \rightarrow aS'$

$S' \rightarrow \alpha_1 / \alpha_2 / \alpha_3$

$w = a\alpha_3$



* $A \rightarrow a\alpha_1 / a\alpha_2 / \beta_1 / \beta_2 \equiv$

$A \rightarrow \beta_1 / \beta_2 / aA'$

$A' \rightarrow \alpha_1 / \alpha_2$

$L = \{\beta_1 + \beta_2 + a(\alpha_1 + \alpha_2)\}$

---

* $A \rightarrow \underline{\alpha_1}\gamma_1 / \underline{\alpha_1}\gamma_2 / \beta_1 / \beta_2 \equiv$

$A \rightarrow \beta_1 / \beta_2 / \alpha_1 A'$

$A' \rightarrow \gamma_1 / \gamma_2$



Topic : Left Factoring

1. $S \rightarrow iEtS / iEtSeS / a$

   $E \rightarrow b$

γ_1 γ_2 β_1

α_1 α_1

$\equiv$

$S \rightarrow a / iEtSS'$

$S' \rightarrow \epsilon / eS$

$E \rightarrow b$

Slide



Topic : Left Factoring

2. $A \rightarrow aA / abB / d \equiv A \longrightarrow d/aA'$

$A' \longrightarrow A/bB$

$B \rightarrow bB / e$

No Common Prefix $\longrightarrow \therefore B \longrightarrow \underbrace{bB/e}_{\text{as it is}}$

Slide



Topic : Left Factoring

3. $S \rightarrow aAd / aB \equiv S \rightarrow aS'$, $S' \rightarrow Ad / B$

$A \rightarrow a / ab \equiv A \rightarrow aA'$, $A' \rightarrow \epsilon / b \equiv A' \rightarrow b / \epsilon$

$B \rightarrow ccd / ddc = B \rightarrow ccd / ddc$

No Common Prefix

Slide



Topic : Left Factoring

4. $S \rightarrow ab / abc / abcd / abcde / af / g \equiv$

$S \rightarrow aS' / g$

$S' \rightarrow b / bc / bcd / bcde / f \equiv S' \rightarrow bS'' / f$

$S'' \rightarrow c / cd / cde / \epsilon \equiv S'' \rightarrow cX / \epsilon$

$X \rightarrow \epsilon / d / de \equiv X \rightarrow dY / \epsilon$

$Y \rightarrow e / \epsilon$

$S \rightarrow aS' / g$

$S' \rightarrow bS'' / f$

$S'' \rightarrow cX / \epsilon$

$X \rightarrow dY / \epsilon$

$Y \rightarrow e / \epsilon$

Slide



Topic : First & Follow

First of X / First(X) : First of X is the set of all terminals that may begin the string derived from X.

After First

Follow of X / Follow(X) : Follow of X is the set of all terminals that may appear immediately on the right side of non-terminal X in some sentential form.

Slide



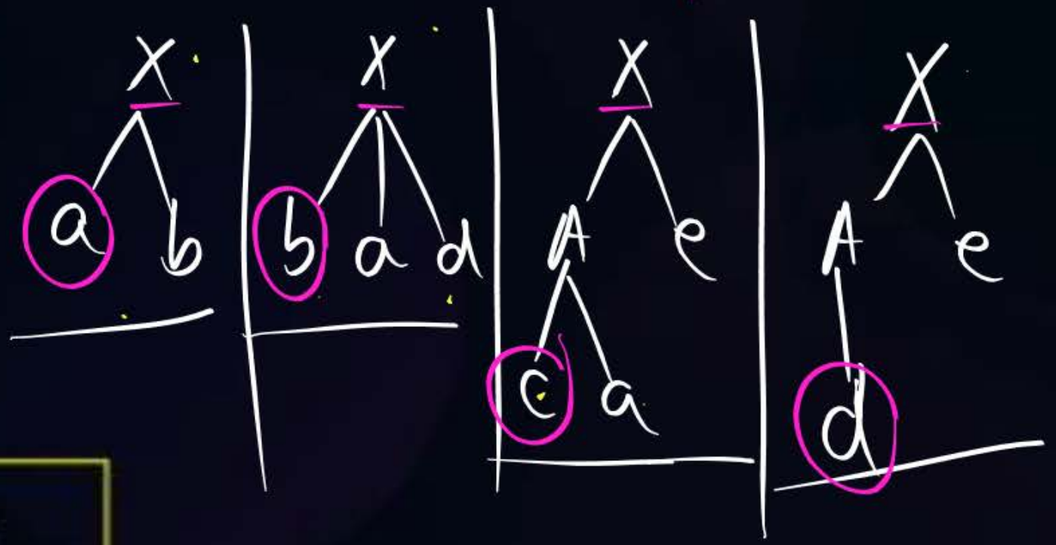
Topic : First

- First of X gives the set of all terminals that may begin the string derived from X.

eg. Gramma

$X \rightarrow ab / bad / Ae$

$A \rightarrow ca / d$



$X \Rightarrow ab$

$\Rightarrow bad$

$\Rightarrow Ae \Rightarrow cae$

$\Rightarrow de$

Strings derivable from X

$first(X) = \{a, b, c, d\}$

Slide



Topic : First

Calculation of First of X

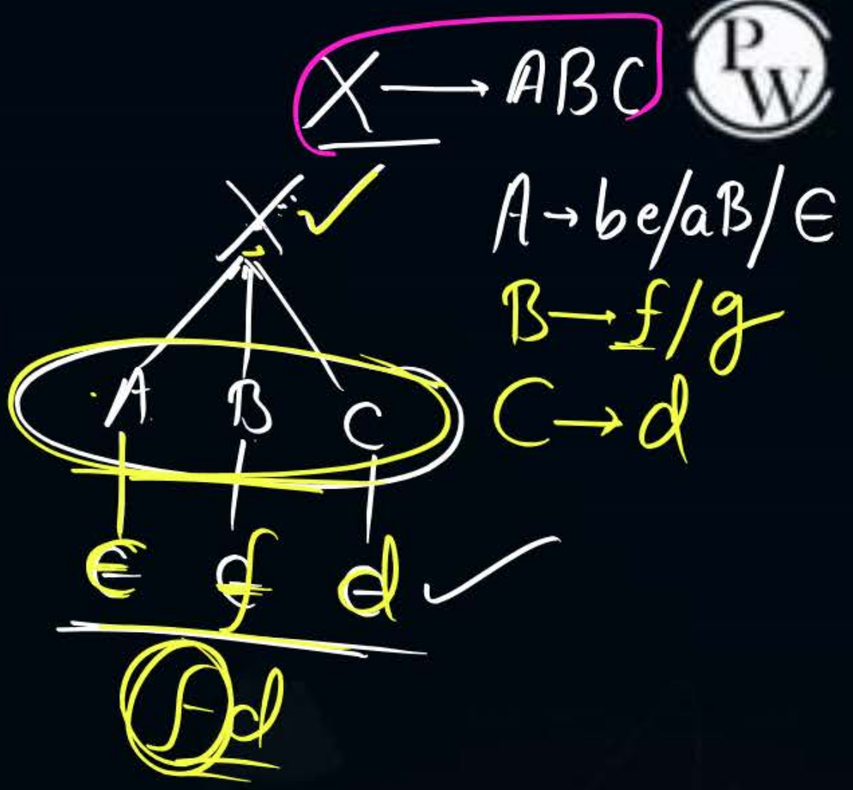
✓ (1) If X is a terminal, then

$$First(X) = X$$

✓ (2) If X is a non-terminal, and

$X \longrightarrow ABC$ is a production in the grammar, then

$$First(X) = First(ABC)$$



Slide



Topic : First

✱ How to Calculate First(ABC) :-

Case ① :- $First(ABC) = First(A)$, if $\epsilon \notin First(A)$

Case ② : $First(ABC) = (First(A) - \{\epsilon\})\ Union\ (First(BC))$, if $\epsilon \in First(A)$

Similarly,

$First(BC) = First(B)$, if $\epsilon \notin First(B)$

$First(BC) = (First(B) - \{\epsilon\})\ Union\ (First(C))$, if $\epsilon \in First(B)$

Similarly First(C)

Note: If ϵ belongs to First(A), as well as First(B), as well as First(C), then ϵ will belong to First(ABC) as well.

Slide

* If $X \longrightarrow ABC / PQR / LMN$ are the productions in the grammar with respect to non-terminal X, then

$$First(X) = First(ABC) \cup First(PQR) \cup First(LMN)$$



Topic : First & Follow

Calculate first of all non-terminals

	First	Follow
$S \rightarrow aAB$	First(S) = First(a AB) = First(a) = {a}	
A → b	First(A) = First(b) = {b}	
B → c	First(B) = First(c) = {c}	

Slide



Topic : First & Follow

	First
$S \rightarrow AB$	First(S) = First(AB) as $\epsilon \in$ First(A) $\therefore$ First(AB) = (First(A) − {ϵ}) ∪ First(B) = ({a, ϵ} − {ϵ}) ∪ {b} = {a, b}
A → a / ϵ	First(A) = First(a) ∪ First(ϵ) = {a} ∪ {ϵ} = {a, ϵ}
B → b	= {b}

Slide



Topic : First & Follow

$X \longrightarrow ABC$ . .

First(X) = First(ABC)

First(α) = First(aABe)

First

X→aABe	First(aABe) = First(a) = {a}
B→c/d	{c} ∪ {d} = {c, d}
A→a	{a}

Slide



Topic : First & Follow

$E \rightarrow TE'$

$E' \rightarrow +TE'/\varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT'/\varepsilon$

$F \rightarrow id/(E)$

= First(TE') = First(T) = {id, (}

First(+TE') ∪ First(ϵ) = {+, ϵ}

First(T) = First(FT') = ∴ First(F) = {id, (}

$\epsilon \notin$ first(F)

First(*FT') ∪ First(ϵ) = {*} ∪ {ϵ} = {*, ϵ}

First(id) ∪ First((E)) = {id} ∪ {(} = {id, (}

Slide



Topic : First & Follow

H.W

S→aBDh

B→cC

C→bc / ε

D→EF

E→g/ ε

F→f/ ε

Slide



Topic : First & Follow

H.W

$S \rightarrow AaAb/BbBa$

$A \rightarrow c/\ \varepsilon$

$B \rightarrow d/\ \varepsilon$

Slide



Topic : First & Follow

H.W

$S \rightarrow (L)/a$

$L \rightarrow SL'$

$L' \rightarrow , SL'/ \varepsilon$

Slide



Topic : First & Follow

H.W

S→ACB/ CbB /Ba

A→da/BC

B→g/ ε

C→h/ ε

Slide



2 mins Summary

Topic Elimination of Left Recursion

Topic Left Factoring

Topic First & Follow

THANK - YOU